

TuringAward Review

The Ledger — Architectural Constitution (v1.3) and Specification (v16.0)

TuringAward — first-principles architecture and algorithm review

14 July 2026 (committee mode; advisory, no veto)

Version note (numbered finding, no conflict absorbed silently). *TA-M-00 [INFO]* cross-document. The invocation names “v1.1” as the authority; the repository’s sole authority at review time is `ledger_manifesto_v1_3.tex` (v1.3, 14 July 2026), whose amendment record carries the v1.1 collateral ruling and everything since. This review is conducted against v1.3 and covers the v1.1 material through v1.3’s text and amendment record. The invocation-versus-repository version gap is recorded here rather than resolved silently; it is a labelling matter for **CONCORDIA** to reconcile, not a substantive conflict.

1 Overall Comment

What the system is. One immutable, append-only, hash-chained log of admitted transactions is the sole canonical record; every other number — balance, valuation, profit and loss, report — is a deterministic fold of that log. Three machines evaluate three arrows: the Event Monitor turns clock and record into events; the Events Executor fires pure stateless contracts that map an event and the current ledger to a proposed transaction; the Transaction Executor is the single writer that admits or refuses. State is the coordinate vector (owned/lent/posted, keyed by agreement) plus three homes (ProductTerms, UnitStatus, PositionState). The system is closed by virtual wallets, so conservation is a paired-leg identity, not a check.

Inventory, classified before naming (§2 of the reviewer’s charter). *Safety:* conservation per (unit, coordinate, agreement) = 0; append-only and hash-chained; one legal writer per fact; consistency of reference; idempotence under the cause-derived *txid*; coverage bounded by possession. *Liveness:* watches arm and fire; obligations discharge, compensate, or default by deadline; corrections and close-outs terminate. *Failure model:* *stated nowhere explicitly* — inferable from the trust boundary (C-5.8) as crash-recovery with omission, duplication and re-ordering *above* the door, and a trusted, un-replicated, presumed-fail-stop writer *at* the door; the writer’s own failure is addressed only obliquely, by the authorised fork (C-12.5). This is recorded as OBSERVATION-zero and drives finding TA-M-01. *Consistency model:* one writer, one total order, sequential apply — trivially serializable, in fact linearizable at the log; bitemporal on two axes (transaction-time prefix, valid-time horizon). *Cost:* replay and every projection are $O(n)$ in log length n , which grows without bound; no checkpoint primitive is fixed. *Representation risk:* quantities are exact integers in minor units (excellent), but the variance-swap accrual computes log-returns $\ln(C_i/C_{i-1})$, and no document pins transcendental arithmetic to a reproducible representation — a live L9 gap (TA-M-04).

What it gets right at first principles. The thesis is correct and cleanly executed: two projections of one record cannot disagree, so internal reconciliation failure is removed at its architectural root rather than mitigated by process. Four choices are genuinely first-principles, not custom. (i) Illegal states are made *unrepresentable*, not merely refused: the product graph gives each lifecycle node a type and each edge a constructor, and there is no un-knock constructor, so a forgotten case is a compile error (Spec Ch. 3, L5). (ii) Conservation is stated at the *finest* grain — per (unit, coordinate, agreement) — and the choice is *forced* by a concrete failure (post 100 under G_1 , receive 100 under G_2 , net zero hides a fully encumbered wallet), which is exactly

how a minimal basis should be argued (Ch. 14, L2/L4). (iii) Purity of contracts makes economic correctness *checkable by recomputation* — re-run the map on the recorded event and compare — and the design honestly separates structural correctness (guaranteed at the door) from economic correctness (detectable, repairable), refusing a pretence of prevention it cannot deliver (Ch. 15, L3). (iv) The executable-check regime treats a never-fired precondition as a *defect*, asserting each branch fires on $\geq 1\%$ of histories under **checkCoverage**; this is the discipline most property suites lack (Claessen & Hughes), and it is the specification’s strongest single feature. Bitemporality on two axes is done correctly, and the conceptual integrity is exceptional: one mind, one document, one vocabulary enforced (Brooks would recognise it; L6).

What canonical system it most resembles. Three, and the resemblance breaks instructively in each. (1) *Datomic* (Hickey): an immutable fact log, the database as a deterministic value folded from it, transaction-time and valid-time as first-class axes. The Ledger is Datomic with a conservation law and an *admission door* — and this is exactly where the resemblance breaks from event sourcing generally: Datomic and event-sourced systems *reinterpret* history by running new code over old events; here contracts are never re-executed on replay, authority lives with the admitted transaction, and reinterpretation is structurally impossible. That door is the real novelty. (2) *A deterministic replicated state machine* (Lamport; Schneider) over a totally-ordered command log — but with the replication removed. The log, the total order, the deterministic apply are all present; the quorum, the consensus, and the fault-tolerance of the writer are not. (3) *Double-entry bookkeeping* (Pacioli, 1494) generalised: the paired-leg move is debit-equals-credit, and the coordinate vector extends it to possession and encumbrance planes. Real-world kin worth studying are *TigerBeetle* (deterministic financial accounting, integer amounts, single-writer but VSR-replicated — the replication this design omits) and *FoundationDB* (whose deterministic simulation testing is precisely the “one engine, two measures” simulated-ledger idea of Spec §10.7).

Where the deepest risks sit. The single trusted writer is the entire trust base, and it is un-replicated: “the ledger’s integrity rests on it and on nothing else” (C-5.8). That sentence is a strength for reasoning and a risk for operation — a crash mid-append, silent disk corruption, or a compromised writer has no quorum to correct it, only the authorised fork, which itself needs a trustworthy replay source. The manifesto never states the writer’s failure model, so the reader cannot tell whether a single-node fail-stop assumption or a replicated one is intended (TA-M-01). Second, “immutable” and “any alteration is detectable” overclaim: hash-chaining is tamper-*evident* relative to an external anchor, not tamper-*proof* against the party that computes the chain; without published or witnessed hashes a compromised writer re-chains undetectably (TA-M-03, L8; this is Haber & Stornetta’s point, and their anchoring is the missing half). Third, “reproducible to the last minor unit” is asserted while the arithmetic that would break it — transcendental and floating-point computation — is never constrained (TA-M-04, L9). Fourth, the single total order is required but its *construction* among concurrent external arrivals is left implicit; in any multi-ingestion deployment that is a latent consensus problem, and FLP bounds what timeouts can achieve (TA-M-05, L1). Fifth, replay and time-travel are $O(n)$ with no constitutional checkpoint discipline, and a naive snapshot would reintroduce the “second store” the design exists to abolish — the tension is real and unaddressed (TA-M-07, L2/L6). None of these is a defect in the thesis; each is a place where a load-bearing assumption is unstated. The formal adequacy of Theorem 14.4 (time travel) and Theorem 14.1 (conservation) is sound on inspection but is a question for **FORMALIS** to certify, not for this reviewer to close.

2 Manifesto: Proposed Improvements

Each proposal names the article, the weakness, one paragraph of proposed text or addition, and the motivating lens. Every proposal is a *proposal*: amendment is the owner’s alone. MAJOR

items carry two candidate remedies from different lenses.

TA-M-01 [MAJOR] L1 concurrency/distribution. **The failure model is implicit.** *Article:* §2 (Commitments) and C-5.8 (trust boundary); C-12.5 (authorised fork). *Weakness:* the constitution never states a failure model. C-5.8 implies one for the untrusted machines (delay, duplication, spurious firing) but never names it (crash-recovery? omission? Byzantine?), and never states what is assumed of the *trusted* Transaction Executor itself: fail-stop, single-node, replicated, or Byzantine-tolerant. An unstated failure model is itself a defect (reviewer charter, Step 0), and here it is load-bearing because every integrity claim rests on the writer. *Proposed addition (a clause under §2 or §5):* “The failure model is stated once and holds throughout. The Event Monitor and the Events Executor are crash-recovery components subject to omission, duplication, and reordering; no failure they commit can corrupt ledger state, only delay it. The Transaction Executor is a trusted, fail-stop single writer; its durability and the integrity of the log are assumptions named here (A-DURABILITY, A-WRITER-HONEST), not properties the fold provides, and record corruption is remedied only by the authorised fork of C-12.5. Byzantine behaviour of the writer is out of scope and named as such.” *Lens:* L1 (safety vs. failure model; Lamport, Schneider). *Remedy A (L1, replicate the writer):* state-machine replication of the writer by Raft or Viewstamped Replication over $2f + 1$ nodes tolerates f crash faults and removes the single point of failure; cost is a consensus round per commit and the operational burden of a cluster — and it changes no arrow, because the apply function is already a deterministic state machine. *Remedy B (L2, keep one writer, harden durability):* retain the single writer but mandate a write-ahead-log-plus-fsync durability discipline (ARIES-style) and external hash anchoring (TA-M-03), so the writer is fail-stop-recoverable and its corruption is externally detectable; cost is far lower operationally but leaves availability a single point. The choice between them is a deployment question the owner supplies (§10 of the project brief), which is exactly why the model must be *named* even if left open.

TA-M-02 [MAJOR] L3 specification/verification. **No liveness commitment.** *Article:* §2 (the eight commitments). *Weakness:* all eight commitments are safety or quality properties (auditability, reproducibility, time-travel, correctness, testability, clarity, order, simulability). Liveness — that watches eventually arm and fire, that obligations eventually discharge, that corrections eventually settle — appears only as a minimum requirement buried in C-14.8 (Obligation liveness), not as a first-class commitment, even though C-5.8 says the untrusted machines “owe” liveness and the specification’s central collateral episode is governed by a leads-to property (Spec §15.6). A constitution whose correctness argument turns on a safety/liveness split (Pnueli 1977; Lamport 1977) should name liveness as a commitment. *Proposed addition:* either a ninth commitment (“**Liveness.** Every obligation the record creates carries a deadline, a discharge test, and a fallback, and by its deadline is discharged, compensated, or declared defaulted; a duty that fails to fire degrades to a visible overdue item, never to silence”), or an explicit liveness clause folded into Correctness (C-2.4), cross-referenced from C-5.8 and C-14.8. *Lens:* L3 (temporal logic; safety is not liveness). *Remedy A (L3, make it a checkable temporal obligation):* state liveness as a bounded-horizon leads-to property with a named fairness hypothesis, exactly as Spec §15.6 already does in TLA⁺-style, and lift that formulation into the constitution’s vocabulary; cost is only editorial. *Remedy B (L7, end-to-end):* frame liveness as an endpoint obligation — the party owning each deadline is named, and non-discharge surfaces as a reconciliation item at the perimeter — which keeps the constitution from promising a liveness the middleware cannot guarantee; cost is that some obligations become perimeter matters rather than internal guarantees, which is honest.

TA-M-03 [MAJOR] L8 security/cryptography. **“Immutable” and “any alteration detectable” overclaim.** *Article:* C-4.8 (the immutable log, “hash-chained so that any alteration of history is detectable”). *Weakness:* a hash chain computed by a single writer is *tamper-evident*

only against an observer holding an independent reference hash; the writer that produces the chain can alter a past entry and recompute every subsequent hash, and no party lacking an external anchor can detect it. The word “immutable” and the phrase “any alteration detectable” therefore assert cryptographic immutability the mechanism does not provide against the one party that matters. *Proposed text (amend C-4.8)*: “The log is append-only and hash-chained, making alteration detectable *by any party holding an externally anchored or independently witnessed hash*. Absent such an anchor the chain is tamper-evident only against third parties, not against the writer; periodic anchoring of the head hash to an external, append-only witness is therefore a stated requirement, not an optional hardening.” *Lens*: L8 (hash chains and linked timestamping; Haber & Stornetta 1991; Merkle 1987). *Remedy A (L8, anchor)*: publish the head hash periodically to an independent append-only witness (a counterparty, an auditor, or a public timestamp service), turning tamper-evidence into a guarantee that bites against the writer; cost is one external write per anchoring interval and a governance owner for the witness. *Remedy B (L8, sign)*: have the writer sign each committed head with a key held under separate control, so alteration requires the signing key as well as the log; cost is key management and the fact that a fully compromised writer holding the key still defeats it — which is why anchoring (A) is the stronger of the two and they compose.

TA-M-04 [MAJOR] L9 numerical computation. Bit-exact reproducibility is claimed but the arithmetic is not constrained. *Article*: C-2.2 (reproducibility “to the last minor unit”) and C-6.4 (contract purity). *Weakness*: purity guarantees the *same inputs give the same outputs on one machine*; it does not guarantee the *same output across machines* when the computation uses floating point or transcendental functions, whose results vary by platform, compiler, and library (IEEE 754 leaves transcendental functions unspecified; Goldberg 1991). The specification’s own variance-swap accrual computes $\ln(C_i/C_{i-1})$ and squares it (Spec §5.6, §7.5, §15), yet claims “any party ... obtains the identical A_{127} .” Purity is necessary and not sufficient; the reproducibility commitment has a hole exactly where a real instrument sits. *Proposed addition (a clause under §2)*: “Every quantity that enters the record is an exact integer in minor units or a value in a declared fixed-point or rational representation with a specified rounding rule; no platform-dependent floating-point result is admitted to the record. Where an instrument’s economics require a transcendental function, the function, its argument reduction, and its rounding are pinned to a named, versioned, reproducible specification (for example a correctly-rounded decimal library), so that reproduction to the last minor unit is a property of the record and not of the platform.” *Lens*: L9 (floating-point determinism; Kahan; Goldberg). *Remedy A (L9, exact representation)*: compute returns and accruals in fixed-point decimal with correctly-rounded elementary functions and a declared rounding mode, so replay is bit-exact by construction; cost is a dependency on a correctly-rounded library and slightly slower arithmetic. *Remedy B (L5, record the result, not the computation)*: treat any transcendental output as a boundary *observation* with provenance — computed once, off the record, and admitted through the observation door like a price — so the fold never recomputes it and cross-platform reproduction is not required; cost is that the number is no longer verifiable by recomputation, only by its provenance, which weakens the economic-correctness oracle for that instrument. The two trade determinism against verifiability, and the owner should choose per computation.

TA-M-05 [MAJOR] L1 concurrency/distribution. The single total order is required but its construction is unspecified. *Article*: C-2.7 (Order) and C-5.7 (the writer refuses non-commuting no-precedence pairs). *Weakness*: C-2.7 forces “the single writer, the single total order,” and C-5.7 has the writer *refuse* an order it cannot justify — but neither says how the total order is *established* among genuinely concurrent external arrivals reaching the boundary from independent sources. On one node the order is whatever the writer sees; the moment there are multiple ingestion points, fixing a single order is a distributed agreement problem, and FLP (Fischer, Lynch, Paterson 1985) says no asynchronous protocol solves it

with both safety and liveness under a single crash. The constitution should not leave this implicit. *Proposed addition (a clause under §5)*: “Establishing the total order is the Transaction Executor’s sole prerogative: all proposals are serialised through the one writer, so the order is the writer’s admission order. Where proposals originate from more than one ingestion point, either they are funnelled through a single serialisation point before the door, or the deployment names the ordering protocol and its failure model as an assumption. The refusal of C-5.7 is a safety backstop, not an ordering construction.” *Lens*: L1 (consensus; FLP; Lamport total order). *Remedy A (L1, single serialisation point)*: funnel all ingestion through one writer, so the order is trivially the writer’s sequence and no consensus is needed; cost is that the writer is a throughput bottleneck and a single point (compounding TA-M-01). *Remedy B (L1, consensus log)*: if throughput or availability forbid a single point, order proposals with a consensus protocol (Raft/VR) and make the committed log the agreed order; cost is a consensus round per commit and the acknowledgement that liveness is now conditional on a quorum, which must be stated, not assumed away by timeouts.

TA-M-06 [MINOR] L5 abstraction/language. **“Cannot be represented” conflates laws with door-checked guards.** *Article*: C-11 (“certain classes of inconsistent or illegal state cannot be represented at all,” listing atomicity, consistency of reference, and writer discipline together). *Weakness*: the specification (Ch. 14) correctly divides the catalogue into *laws* that hold by construction (conservation, no un-knock constructor — genuinely unrepresentable) and *guards* the Transaction Executor computes on every admission (coverage, consistency of reference, writer discipline — *refused*, not unrepresentable). The constitution lists them under one heading, so “what cannot be represented cannot break” reads as covering door-enforced guards, whose correctness depends on the door’s implementation being right. *Proposed text (amend C-11)*: distinguish the two classes explicitly — “Some illegal states are unrepresentable (no type constructs them); others are refused at the single door on every admission. The first class cannot break; the second cannot break *while the door is correct*, which is why the door is the one trusted component.” *Lens*: L5 (types as theorems vs. runtime checks). This is a precision matter that **CONCORDIA** should reconcile against the spec’s own Ch. 14 wording, which already draws the line.

TA-M-07 [MINOR] L2 transactions/data. **Replay is $O(n)$ with no checkpoint discipline, and the “no second store” rule forbids the obvious fix.** *Article*: C-2.3 (time travel “at any time, forever”) and C-1.4/C-3.5 (every view a fold of the log). *Weakness*: reconstructing any past state, and computing any projection, folds a prefix whose length grows without bound; at scale this is the difference between a system that answers in milliseconds and one that does not (Gray’s five-minute rule; the memory hierarchy is the computer). A snapshot is the standard remedy, but a naive materialised snapshot is a “second store” the design abolishes (C-1.2). The constitution should say that a checkpoint is admissible precisely when it is a pure fold provably equal to replay — a cached read, not a rival source. *Proposed addition*: “A checkpoint of the folded state at a log position is admissible as a materialised projection when it is a pure function of the prefix and is discardable and rebuildable by replay without loss; it is a cached read of the log, never a second source of truth. The homes of §7 are exactly such checkpoints, and any state snapshot is held to the same discipline.” *Lens*: L2 (materialised views; recovery). This makes the homes’ status (already “materialised projections” in §7) the general rule, and lets an implementation checkpoint without violating the one-record principle.

TA-M-08 [INFO] L5 clarity. **Two “single non-record input” claims sit in tension.** *Article*: C-3.7 (“the clock is the single input that is not on the record”) and C-2.8 (the simulation seed is “the one non-record input”). *Weakness*: taken literally the two sentences each claim uniqueness for a different input. They are reconcilable — the clock is the only non-record input to *production*, the seed the only one to a *simulated* path, and the seed is itself recorded — but a

reader meeting both is briefly told there are two “only” non-record inputs. *Proposed text*: qualify each (“the single non-record input to production” / “the single non-record input to a simulated path, itself recorded”). *Lens*: L5 (state each thing once, precisely). A one-word edit; flagged for CONCORDIA.

TA-M-09 [MINOR] L3 specification/verification. Economic-correctness detection has no liveness obligation. *Article*: C-13.2–C-13.3 (economic correctness “made checkable by recomputation ... detection is after the fact”). *Weakness*: the recomputation oracle is sound, but the constitution guarantees only that a discrepancy is *checkable*, not that it is *checked* within any bound. A wrong-but-well-formed transaction can stand indefinitely unless someone runs the oracle. Detection is a liveness obligation (ties to TA-M-02) and should be named as one. *Proposed text (amend C-13.3)*: “Economic verification is not only possible but owed: every recorded transaction is recomputed and compared within a declared horizon, and an un-recomputed transaction beyond that horizon is a visible overdue item, never silent trust.” *Lens*: L3 (a property defended only in prose is not enforced). This is the audit analogue of the obligation-liveness rule and closes the gap between “checkable” and “checked.”

3 Related Work

Canonical sources over surveys; each entry gives the contribution in one sentence and the Ledger part it speaks to.

1. **Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System” (CACM 1978).** Happened-before, logical clocks, and the state-machine approach to consistency. Grounds the single total order and the map-then-fold as a deterministic state machine over an ordered command log (C-2.7, Spec Ch. 2).
2. **Schneider, “Implementing Fault-Tolerant Services Using the State Machine Approach” (ACM Comput. Surv. 1990).** A deterministic state machine driven by a totally-ordered request log, replicated for fault tolerance. The Ledger *is* this state machine with the replication removed — the reference for TA-M-01’s Remedy A.
3. **Fischer, Lynch & Paterson, “Impossibility of Distributed Consensus with One Faulty Process” (JACM 1985).** No asynchronous protocol achieves both safety and liveness under a single crash. Bounds what the single-total-order construction can promise once ingestion is distributed (TA-M-05).
4. **Ongaro & Ousterhout, “In Search of an Understandable Consensus Algorithm” (Raft, USENIX ATC 2014).** A replicated commit-log with an elected leader. The concrete way to replicate the single writer and agree the log order (TA-M-01, TA-M-05).
5. **Lamport, “The Part-Time Parliament” (Paxos, TOCS 1998).** The foundational asynchronous consensus result behind replicated logs. The theory beneath Raft/VR for the un-replicated writer.
6. **Gray & Reuter, *Transaction Processing: Concepts and Techniques* (1992); Gray, “The Transaction Concept” (1981).** ACID, write-ahead logging, and recovery. The admission door is a single-writer commit point, and the writer needs this recovery discipline (TA-M-01 Remedy B).
7. **Mohan et al., “ARIES” (ACM TODS 1992).** Write-ahead logging with repeating history and crash recovery. The durability model an un-replicated writer must adopt to be fail-stop-recoverable (TA-M-01, TA-M-03).

8. **Bernstein, Hadzilacos & Goodman, *Concurrency Control and Recovery in Database Systems* (1987).** Serializability theory. Explains why a single sequential writer is trivially serializable, and what is given up relative to concurrent execution (Spec Ch. 4, §14).
9. **Hickey / *Datomic*, “The Database as a Value.”** An immutable append-only fact log with transaction-time and valid-time axes and the database as a deterministic fold. The closest real-world kin; the resemblance breaks at the admission door and conservation (Spec Ch. 2, §2.5).
10. **Snodgrass, *Developing Time-Oriented Database Applications in SQL* (2000).** The canonical treatment of bitemporal data — transaction time versus valid time. Directly underwrites the two-axis replay of Spec §2.5 and Theorem 14.4.
11. **Pnueli, “The Temporal Logic of Programs” (FOCS 1977); Lamport, “Proving the Correctness of Multiprocess Programs” (1977).** The safety/liveness distinction. The frame for the missing liveness commitment (TA-M-02) and for §15.6’s leads-to model.
12. **Lamport, *Specifying Systems: The TLA⁺ Language and Tools* (2002).** Temporal-logic specification with a model checker. Spec §15.6 writes its safety/liveness model in TLA⁺-style illustration; naming the tool and machine-checking it would strengthen it.
13. **Claessen & Hughes, “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs” (ICFP 2000).** Property-based testing with generators and shrinkers. The direct ancestor of Spec Ch. 15, whose “zero firings is a defect” rule (`checkCoverage`) is the improvement most suites omit.
14. **FoundationDB deterministic simulation testing (Zhou et al., and the project’s simulation talks).** A single deterministic engine run under an adversarial scheduler to refute invariants, and re-run to serve requests. Exactly the “one engine, two measures” simulated ledger of Spec §10.7 and §15.
15. **Pacioli, *Summa de Arithmetica* (1494); modern: *TigerBeetle* financial accounting database.** Double-entry as debit-equals-credit; TigerBeetle as a deterministic, integer-amount, single-writer *but VSR-replicated* accounting state machine. The origin of the paired-leg conservation law and a production system showing the replication the Ledger omits (Ch. 3, Ch. 14; TA-M-01).
16. **Haber & Stornetta, “How to Time-Stamp a Digital Document” (J. Cryptology 1991); Merkle, “A Digital Signature Based on a Conventional Encryption Function” (1987).** Linked timestamping and hash trees, with external anchoring as what makes a chain tamper-evident against its own producer. The precise basis for TA-M-03.
17. **Goldberg, “What Every Computer Scientist Should Know About Floating-Point Arithmetic” (ACM Comput. Surv. 1991); IEEE 754.** Floating-point behaviour and the unspecified status of transcendental functions across platforms. The reason C-2.2’s “to the last minor unit” needs TA-M-04.

4 Declaration

Lenses applied. *L1 (concurrency/distribution):* the single total order, the un-replicated trusted writer, the failure model, order construction (TA-M-01, TA-M-05). *L2 (transactions/data):* the log as a single-writer commit point, trivial serializability, replay cost and checkpoints (TA-M-07). *L3 (specification/verification):* safety vs. liveness, the TLA⁺-style

model, the missing liveness commitment, detection liveness (TA-M-02, TA-M-09). *L4 (algorithms/complexity)*: $O(n)$ replay and projection; no NP-hard subproblem found — netting and close-out are projections, not optimisations (noted, no finding). *L5 (abstraction/language)*: illegal states unrepresentable via the product graph; law vs. door-checked guard; the two “single non-record input” claims (TA-M-06, TA-M-08). *L6 (systems/architecture)*: exceptional conceptual integrity (one mind, one document, one vocabulary); the second-system risk is low; the missing snapshot discipline is the one place Lampson would ask “what would you delete?” (TA-M-07). *L7 (networks/end-to-end)*: at-least-once emission plus idempotent apply places reliability at the endpoint (the door), which is the end-to-end argument done right; informs TA-M-02 Remedy B. *L8 (security/cryptography)*: hash-chain tamper-evidence vs. tamper-proofness; the compromised-writer adversary; anchoring and signatures (TA-M-03). *L9 (numerical computation)*: integer minor units (correct) but unconstrained transcendental arithmetic in the variance-swap accrual (TA-M-04).

Lens dismissed. *L10 (learning/inference)*: no learned component is load-bearing for any safety property; strategies and generators are declared, deterministic models confined outside the fold, with their seed recorded — there is nothing learned near an invariant to challenge.

Addressed by name (committee mode). Formal adequacy of Theorem 14.1 (conservation), Theorem 14.4 (time travel), and the injectivity assumption on H in the *txid* derivation is for **FORMALIS** to certify. Cross-document consistency questions — the invocation-vs-repository version gap (TA-M-00), the law-vs-guard wording (TA-M-06), and the two “single non-record input” sentences (TA-M-08) — are for **CONCORDIA**. Domain-specific depth on the collateral regimes and close-out algebra is deferred to the resident specialists (MINSKY, the SBL specialist); this review flags the architecture, not the desk economics.

Confidence: MEDIUM–HIGH. The thesis, the invariant catalogue, and the executable-check regime are strong and internally coherent, and the findings are gaps in stated assumptions rather than errors in the design. *Single biggest unknown*: the deployment topology. Both documents deliberately leave repository layout, build, and run to the owner (project brief §10), so whether the trusted writer is a single fail-stop node or a replicated cluster is undetermined — and the severity of TA-M-01, TA-M-03, and TA-M-05 turns on that choice. Named, not assumed.